EXP-4

(8)

5. Write a program for optimal merge patterns.

Experiment:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  char data;
  int freq;
  struct Node *left, *right;
};

struct Node* createNode(char data, int freq) {
  struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
  newNode->data = data;
  newNode->freq = freq;
  newNode->left = newNode->right = NULL;
  return newNode;
}

void printHuffmanCodes(struct Node* root, int arr[], int top) {
  if (root->left) {
    arr[top] = 0;
    printHuffmanCodes(root->left, arr, top + 1);
  }

  if (root->right) {
    arr[top] = 1;
    printHuffmanCodes(root->right, arr, top + 1);
  }

  if (!root->left && !root->right) {
    printf("%c: ", root->data);
    for (int i = 0; i < top; i++)
      printf("%d", arr[i]);
    printf("\n");
```

Branch

Semester 4th

Subject

```
  }
}

void buildHuffmanTree(char data[], int freq[], int size) {
  struct Node* left, * right, * top;

  struct Node* minHeap[size];

  for (int i = 0; i < size; i++)
    minHeap[i] = createNode(data[i], freq[i]);

  for (int i = 0; i < size - 1; i++) {
    left = minHeap[i];
    right = minHeap[i + 1];

    top = createNode('$', left->freq + right->freq);
    top->left = left;
    top->right = right;

    minHeap[i + 1] = top;
  }

  int arr[100], topIndex = 0;
  printHuffmanCodes(minHeap[0], arr, topIndex);
}

int main() {
  char data[] = {'a', 'b', 'c', 'd', 'e', 'f'};
  int freq[] = {5, 9, 12, 13, 16, 45};
  int size = sizeof(data) / sizeof(data[0]);

  buildHuffmanTree(data, freq, size);

  return 0;
}
```

Branch

Semester 4th

6. Write a program for Huffman coding.

Experiment:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  char data;
  int freq;
  struct Node *left, *right;
};

struct Node* createNode(char data, int freq) {
  struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
  newNode->data = data;
  newNode->freq = freq;
  newNode->left = newNode->right = NULL;
  return newNode;
}

void printHuffmanCodes(struct Node* root, int arr[], int top) {
  if (root->left) {
    arr[top] = 0;
    printHuffmanCodes(root->left, arr, top + 1);
  }

  if (root->right) {
    arr[top] = 1;
    printHuffmanCodes(root->right, arr, top + 1);
  }

  if (!root->left && !root->right) {
    printf("%c: ", root->data);
    for (int i = 0; i < top; i++)
      printf("%d", arr[i]);
```

Branch

Semester 4th

```
    printf("\n");
  }
}

void buildHuffmanTree(char data[], int freq[], int size) {
  struct Node* left, * right, * top;

  struct Node* minHeap[size];

  for (int i = 0; i < size; i++)
    minHeap[i] = createNode(data[i], freq[i]);

  for (int i = 0; i < size - 1; i++) {
    left = minHeap[i];
    right = minHeap[i + 1];

    top = createNode('$', left->freq + right->freq);
    top->left = left;
    top->right = right;

    minHeap[i + 1] = top;
  }

  int arr[100], topIndex = 0;
  printHuffmanCodes(minHeap[0], arr, topIndex);
}

int main() {
  char data[] = {'a', 'b', 'c', 'd', 'e', 'f'};
  int freq[] = {5, 9, 12, 13, 16, 45};
  int size = sizeof(data) / sizeof(data[0]);

  buildHuffmanTree(data, freq, size);

  return 0;
}
```

Branch

Semester 4th

Exp-5 (10)

7. Write a program for minimum spanning trees using Kruskal's algorithm.

Experiment:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
  int u, v, weight;
} Edge;

int parent[100], rank[100];

int find(int i) {
  if (parent[i] == i)
    return i;
  return parent[i] = find(parent[i]);  // Path compression
}

void union_sets(int u, int v) {
  int root_u = find(u);
  int root_v = find(v);
  if (root_u != root_v) {
    if (rank[root_u] > rank[root_v])
      parent[root_v] = root_u;
    else if (rank[root_u] < rank[root_v])
      parent[root_u] = root_v;
    else {
      parent[root_v] = root_u;
      rank[root_u]++;
    }
  }
}

int compare(const void *a, const void *b) {
  return ((Edge*)a)->weight - ((Edge*)b)->weight;
```

Branch

```
}

void kruskal(int n, Edge edges[], int e) {
  for (int i = 0; i < n; i++)
    parent[i] = i, rank[i] = 0;

  qsort(edges, e, sizeof(Edge), compare);

  printf("Minimum Spanning Tree edges:\n");

  int mst_weight = 0;
  for (int i = 0; i < e; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    if (find(u) != find(v)) {
      union_sets(u, v);
      printf("Edge (%d, %d) with weight %d\n", u, v, edges[i].weight);
      mst_weight += edges[i].weight;
    }
  }
  printf("Total weight of MST: %d\n", mst_weight);
}

int main() {
  int n = 4, e = 5;
  Edge edges[] = {
    {0, 1, 10}, {0, 2, 6}, {0, 3, 5}, {1, 3, 15}, {2, 3, 4}
  };

  kruskal(n, edges, e);

  return 0;
}
```

Branch

4th

12

EXP-6

8. Write a program for minimum spanning trees using Prim's algorithm.

Experiment:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int graph[MAX][MAX], parent[MAX], key[MAX], visited[MAX];

void prim(int n) {
  for (int i = 0; i < n; i++) {
    key[i] = INT_MAX;
    visited[i] = 0;
  }
  key[0] = 0;
  parent[0] = -1;

  for (int count = 0; count < n-1; count++) {
    int min = INT_MAX, u;
    for (int v = 0; v < n; v++) {
      if (!visited[v] && key[v] < min) {
        min = key[v];
        u = v;
      }
    }

    visited[u] = 1;

    for (int v = 0; v < n; v++) {
      if (graph[u][v] && !visited[v] && graph[u][v] < key[v]) {
```

Branch

Semester 4th

```
            key[v] = graph[u][v];
            parent[v] = u;
        }
    }
}

printf("Minimum Spanning Tree edges:\n");
for (int i = 1; i < n; i++) {
    printf("Edge (%d, %d) with weight %d\n", parent[i], i,
graph[i][parent[i]]);
}
}

int main() {
    int n = 4;

    int graph[4][4] = {
        {0, 10, 6, 5},
        {10, 0, 15, 0},
        {6, 15, 0, 4},
        {5, 0, 4, 0}
    };

    prim(n);

    return 0;
}
```

Branch

Exp-7

9. Write a program for single sources shortest path algorithm.

Experiment:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int graph[MAX][MAX], dist[MAX], visited[MAX], parent[MAX];

void dijkstra(int n, int start) {
  for (int i = 0; i < n; i++) {
    dist[i] = INT_MAX;
    visited[i] = 0;
  }
  dist[start] = 0;

  for (int count = 0; count < n - 1; count++) {
    int min = INT_MAX, u;
    for (int v = 0; v < n; v++) {
      if (!visited[v] && dist[v] < min) {
        min = dist[v];
        u = v;
      }
    }

    visited[u] = 1;

    for (int v = 0; v < n; v++) {
      if (graph[u][v] && !visited[v] && dist[u] + graph[u][v] < dist
        dist[v] = dist[u] + graph[u][v];
```

```
            parent[v] = u;
        }
      }
    }

    printf("Shortest path from vertex %d:\n", start);
    for (int i = 0; i < n; i++) {
      if (dist[i] != INT_MAX)
        printf("Distance to %d: %d\n", i, dist[i]);
      else
        printf("No path to %d\n", i);
    }
}

int main() {
    int n = 4;
    int graph[4][4] = {
      {0, 10, 0, 5},
      {10, 0, 15, 0},
      {0, 15, 0, 10},
      {5, 0, 10, 0}
    };

    dijkstra(n, 0);  // Start from vertex 0

    return 0;
}
```